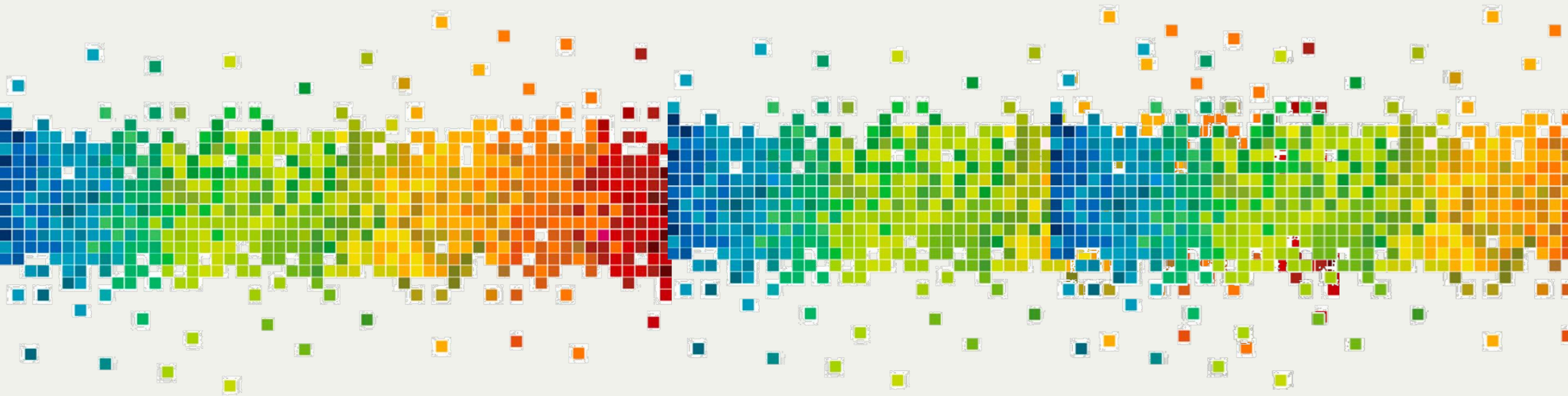


ANALISIS ALGORITMA

Kelas-Kelas Efisiensi Dasar Algoritma



Pemateri:

Nur Azizah Eka Budiarti, M.Kom



Efisiensi waktu sejumlah besar algoritma dikelompokkan ke dalam beberapa **kelas efisiensi dasar**.

Konstanta pengali **akan diabaikan**
dalam kelas efisiensi dasar.

Mungkin saja algoritma efisiensi buruk **memproses lebih cepat** dibanding algoritma efisiensi baik.

Contoh:

Algoritma A = n^3 dan algoritma B = $10^6 n^2$. Kecuali $n > 10^6$, algoritma B memproses lebih cepat dibanding algoritma A sehingga konstanta diabaikan.

Kelas efisiensi dasar ini dibagi menjadi dua, yaitu **kelas algoritma polynomial** dan **kelas algoritma eksponensial**.

Kelas Efisiensi Dasar Algoritma

Kelompok Algoritma	Nama
$O(1)$	Konstan
$O(\log n)$	Logaritmik
$O(n)$	Linear
$O(n \log n)$	$n \log n$
$O(n^2)$	Kuadratik
$O(n^*)$	Kubik
$O(2^n)$	Eksponensial
$O(n!)$	Faktorial



Urutan
Spektrum
secara
ascending

Kelas Efisiensi Dasar Algoritma

$\log n$	n	$n \log n$	n^4	n^5	4^n	$n!$
0	1	0	1	1	2	1
1	2	2	4	8	4	2
2	4	8	16	64	16	24
3	8	24	64	512	256	362880
4	16	64	256	4096	65536	20922789888000
5	32	160	1024	32768	4294967296	Too Big

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^+) < \dots < O(2^n) < O(n!)$$

Algoritma Polinomial

Algoritma Eksponensial

Kelas Efisiensi Dasar: $O(1)$

Penjelasan:

- 1 Algoritma dengan efisiensi waktu terbaik.
- 2 Waktu pelaksanaan algoritamanya adalah tetap dan tidak bergantung pada ukuran masukan.

Kelas Efisiensi Dasar: $O(1)$

```
procedure tukar(var a: integer; var b: integer)

var
    temp: integer;

begin
    temp := a;
    a := b;
    b := temp;
end;
```

Efisiensi Waktu:

$$T(n) = 3 = O(1)$$

Kelas Efisiensi Dasar: $O(\log n)$

Penjelasan:

- 1 Merupakan hasil pemotongan ukuran problem dengan faktor konstan pada tiap iterasi algoritma
- 2 Mentransformasikan persoalan besar menjadi beberapa persoalan kecil yang berukuran sama.

Kelas Efisiensi Dasar: $O(\log n)$

3	7	12	15	29
[1]	[2]	[3]	[4]	[5]
la		k	lb	
Left Side			Right Side	

Binary search membagi persoalan pencarian dengan konstan sehingga masuk ke dalam kelas $O(\log n)$

Kelas Efisiensi Dasar: $O(n)$

Penjelasan:

- 1 Merupakan algoritma yang pelaksanaan waktunya linear.
- 2 Terjadi pada algoritma yang setiap elemen masukannya dikenai proses yang sama.

Kelas Efisiensi Dasar: $O(n)$



```
for i:= 1 to maks_array do  
begin  
    array[i]:=array[i]+1;  
end;
```

Efisiensi Waktu:

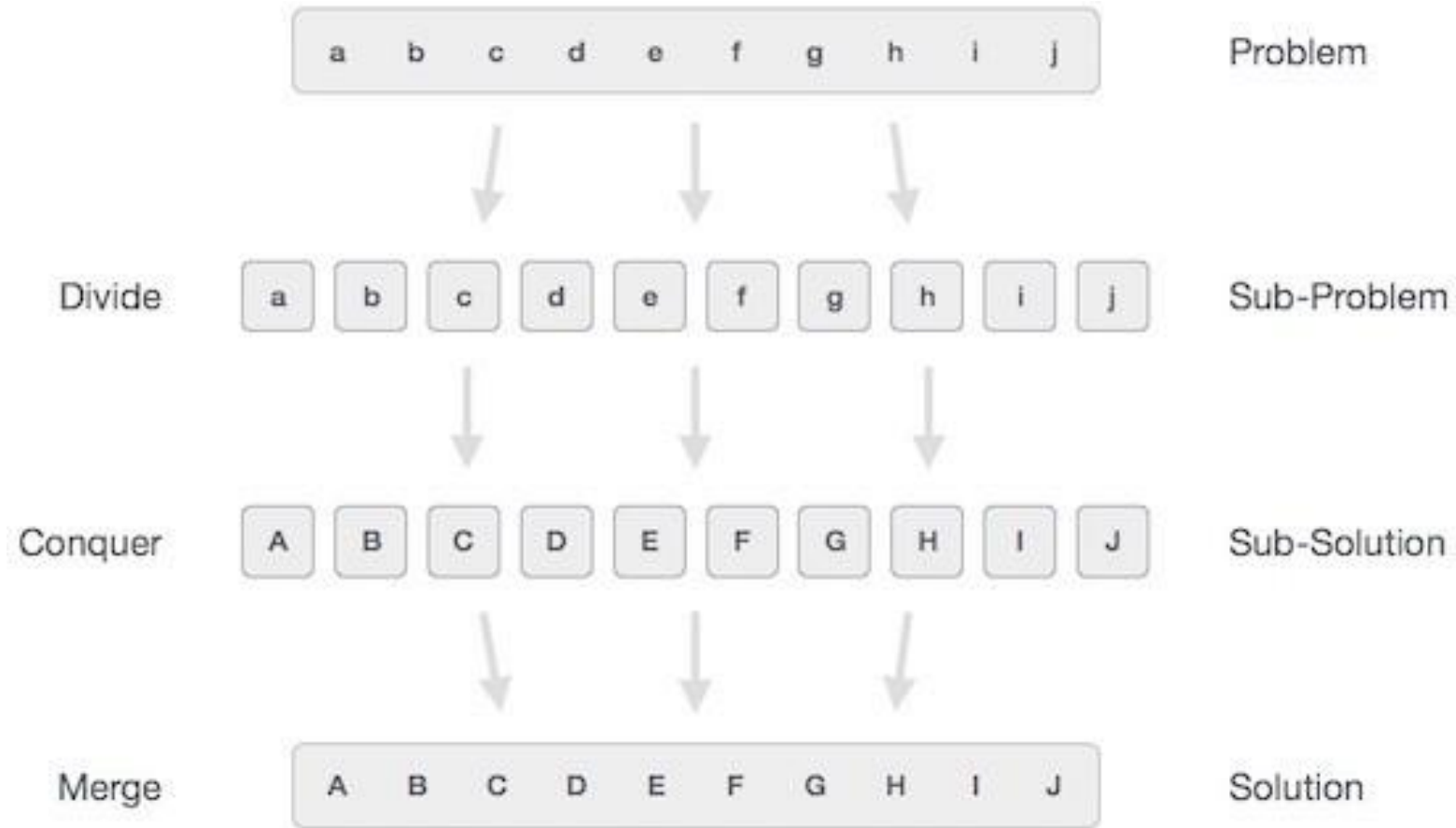
$$T(n) = n = O(n)$$

Kelas Efisiensi Dasar: $O(n \log n)$

Penjelasan:

- 1 Merupakan waktu pelaksanaan algoritma yang memecahkan masalah besar menjadi beberapa masalah kecil, menyelesaikan secara independent, dan menggabungkan solusinya.
- 2 Strategi **Divide and Conquer** memiliki kompleksitas asimptotik ini.

Kelas Efisiensi Dasar: $O(n \log n)$



Kelas Efisiensi Dasar: $O(n^2)$

Penjelasan:

- 1 Hanya praktis digunakan untuk persoalan berukuran kecil.
- 2 Ada dua buah perulangan bersarangan terhadap aksi yang dilakukan.

Kelas Efisiensi Dasar: $O(n^2)$



```
for i:= 1 to jumlah_bintang do
begin
    for j:= 1 to i do
    begin
        write('*');
    end;
end;
```

Efisiensi Waktu:

$$T(n) = n^2 = O(n^2)$$

Kelas Efisiensi Dasar: $O(n^3)$

Penjelasan:

- 1 Hanya praktis digunakan untuk persoalan berukuran kecil.
- 2 Ada tiga buah perulangan bersarangan terhadap aksi yang dilakukan.

Kelas Efisiensi Dasar: $O(n^3)$

```
for i:= 1 to maks_array do
begin
  for j:= 1 to maks_array do
  begin
    for k:= 1 to maks_array do
    begin
      array[i,j,k]:=array[i,j,k]+1;
    end;
  end;
end;
end;
```

Efisiensi Waktu:

$$T(n) = n^3 = O(n^3)$$

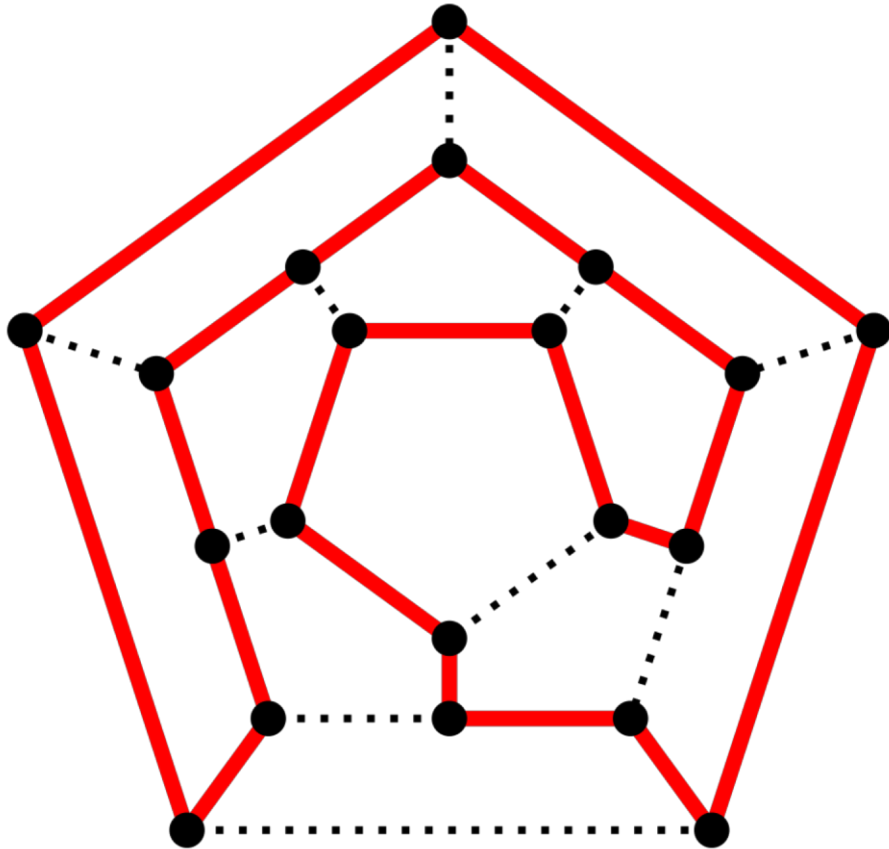
Kelas Efisiensi Dasar: $O(2^n)$

Penjelasan:

- 1 Terjadi pada strategi algoritma **Brute Force**
- 2 Salah satu contohnya adalah pencarian sirkuit Hamilton pada suatu graph.

Kelas Efisiensi

Dasar: $O(2^n)$



python

```
def is_valid(v, pos, path, graph):
    # Cek apakah simpul terhubung dengan simpul sebelumnya
    if graph[path[pos - 1]][v] == 0:
        return False
    # Cek apakah simpul sudah pernah dikunjungi
    if v in path:
        return False
    return True

def hamiltonian_cycle_util(graph, path, pos):
    # Basis: jika semua simpul sudah dikunjungi
    if pos == len(graph):
        # Cek apakah ada sisi kembali ke simpul awal (membentuk sirkuit)
        if graph[path[pos - 1]][path[0]] == 1:
            return True
        else:
            return False

    # Coba berbagai simpul berikutnya
    for v in range(1, len(graph)):
        if is_valid(v, pos, path, graph):
            path[pos] = v
            if hamiltonian_cycle_util(graph, path, pos + 1):
                return True
            path[pos] = -1 # Backtracking
    return False

def hamiltonian_cycle(graph):
    path = [-1] * len(graph)
    path[0] = 0 # Mulai dari simpul 0

    if not hamiltonian_cycle_util(graph, path, 1):
        print("Sirkuit Hamilton tidak ditemukan.")
        return False

    print("Sirkuit Hamilton ditemukan:")
    print(" -> ".join(map(str, path + [path[0]])))
    return True
```

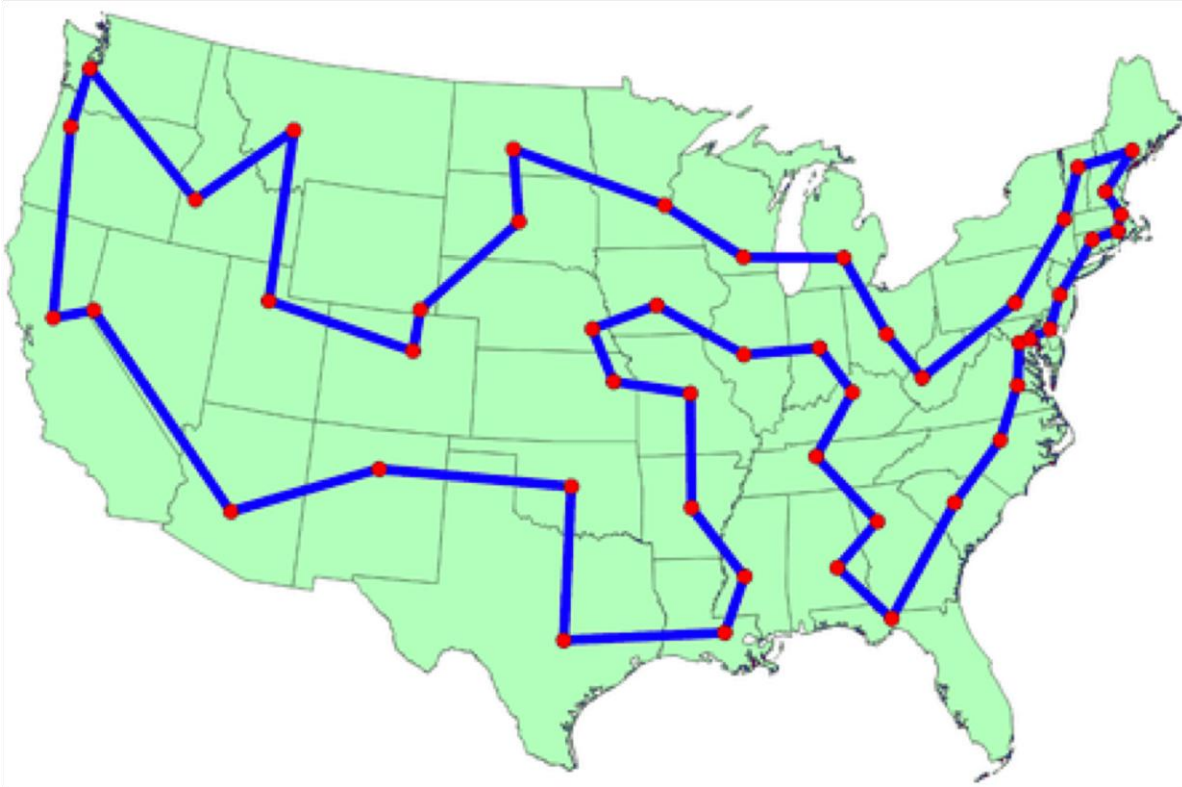
Suatu graph akan dikatakan sirkuit Hamilton jika terbentuk lintasan yang melewati setiap verteks **tepat satu kali** (kecuali verteks asal)

Kelas Efisiensi Dasar: $O(n!)$

Penjelasan:

- 1 Terjadi pada algoritma yang memproses setiap masukannya dan menghubungkan dengan $n-1$ masukan lainnya.
- 2 Contoh paling umum terjadi pada kasus Travelling Salesman Problem.

Kelas Efisiensi Dasar: $O(n!)$



Travelling Salesman Problem mencari jarak minimum yang melewati semua titik dan kembali ke titik asal

Kasus yang memiliki waktu polinomial (derajat rendah) untuk kasus terburuk dianggap merupakan **algoritma yang mangkus.**

Tugas 3

Buktikan Code:

python

```
def is_valid(v, pos, path, graph):
    # Cek apakah simpul terhubung dengan simpul sebelumnya
    if graph[path[pos - 1]][v] == 0:
        return False
    # Cek apakah simpul sudah pernah dikunjungi
    if v in path:
        return False
    return True

def hamiltonian_cycle_util(graph, path, pos):
    # Basis: jika semua simpul sudah dikunjungi
    if pos == len(graph):
        # Cek apakah ada sisi kembali ke simpul awal (membentuk sirkuit)
        if graph[path[pos - 1]][path[0]] == 1:
            return True
        else:
            return False

    # Coba berbagai simpul berikutnya
    for v in range(1, len(graph)):
        if is_valid(v, pos, path, graph):
            path[pos] = v
            if hamiltonian_cycle_util(graph, path, pos + 1):
                return True
            path[pos] = -1 # Backtracking
    return False

def hamiltonian_cycle(graph):
    path = [-1] * len(graph)
    path[0] = 0 # Mulai dari simpul 0

    if not hamiltonian_cycle_util(graph, path, 1):
        print("Sirkuit Hamilton tidak ditemukan.")
        return False

    print("Sirkuit Hamilton ditemukan:")
    print(" -> ".join(map(str, path + [path[0]])))
    return True
```

Program Hamilton -> 2^n

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <limits>

using namespace std;

// Fungsi untuk menghitung biaya minimum jalur TSP
int travelingSalesman(vector<vector<int>> &cost) {
    int numNodes = cost.size();
    vector<int> nodes;

    // Masukkan semua simpul/kota kecuali titik awal (simpul 0)
    for (int i = 1; i < numNodes; i++) {
        nodes.push_back(i);
    }

    int minCost = INT_MAX;

    // Lakukan permutasi pada sisa simpul
    do {
        int currentCost = 0;
        int currentNode = 0;

        // Hitung total biaya perjalanan
        for (int i = 0; i < nodes.size(); i++) {
            currentCost += cost[currentNode][nodes[i]];
            currentNode = nodes[i];
        }

        // Tambahkan biaya kembali ke simpul awal (0)
        currentCost += cost[currentNode][0];

        // Simpan biaya minimum
        minCost = min(minCost, currentCost);
    } while (next_permutation(nodes.begin(), nodes.end()));

    return minCost;
}

int main() {
    // Matriks bobot/jarak antar simpul (graf lengkap)
    vector<vector<int>> cost = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    int minPathCost = travelingSalesman(cost);

    cout << "Biaya minimum perjalanan TSP adalah: " << minPathCost << endl;

    return 0;
}
```

- Program Travelling Sales Man -> $n!$

